Live Region Admission

Residuals, Dependencies and Interference

Mettapedia Working Notes

31 October 2026

Abstract

Six Lean modules place optimization admission at the current request of an indexed computation protocol, rather than at an outer source spelling. Certified partial workers, adaptive read planners and validated code tickets compose with the existing cursor coalgebra. Arbitrary bounded clients retain their replies, continuation and provider state. Interleaved environment changes and semantic-adaptive scheduling preserve complete decoded receipts. A real UInt64 arithmetic worker demonstrates entry after unbounded execution, exit on overflow and later re-entry. These are logical execution laws, not a source-to-native compiler, a native concurrency proof or a speed measurement.

1 The current request is the admission boundary

An indexed polynomial supplies request shapes, typed replies and the capability index following each reply. The existing `Cursor.Provider` supplies a state and one response transition. The existing coalgebraic client exposes either a return or a request with reply-dependent children. Consequently, the client can be adaptive or recursive; a finite scheduler allowance need not imply client termination. No second client syntax is introduced.

For a reference provider M, a partial worker attempts

$$A(s,q) \in \mathrm{Option}\left(\sum_{r:\mathrm{Reply}(q)} \mathrm{State}(\mathrm{next}(q,r))\right).$$

The local obligation is that $A(s,q) = \mathrm{some}(r,s')$ implies $(r,s') = M(s,q)$. Refusal means use the reference operation at the unchanged input, not return an empty answer, false, failure or exhaustion. `LiveRegionAdmission` constructs the mixed provider and a genuine protocol homomorphism to M. The existing client theorem then preserves every bounded client outcome, including the actual residual control.

A portfolio tries certified workers in order. Its admitted set is exactly the union of the workers' admitted sets; adding candidates cannot remove an existing admission. Portfolio composition is associative at the attempt level. This says nothing about which ordering is fastest. Planning, refusals, conversion and validation may all incur physical costs.

The important consequence is temporal: a client can begin outside a region, obtain new bindings or values, and request a later operation inside it. Admission never needs to classify the entire initial program as one fragment. The source frontend must still prove that its operations and observation boundaries lower to this protocol.

2 A region decision is not a reusable code certificate

A planner is a pure adaptive `ReadOnlyQuery.Program` over a state view. It returns optional executable code. Execution has its own local proof against the current reference transition. A prepared ticket stores the code selection and complete ordered read certificate, with a proof-only origin witness. The runtime data need not retain that origin state.

Tickets are indexed by the exact request and planner. On use, the live view is compared with every captured read. Agreement licenses the old selection; disagreement runs planning on the current view. The theorem `cached_attempt_eq_live` equates the resulting attempt with fresh planning. This covers both cached success and cached refusal. In particular, a formerly ineligible request can become eligible after its dependencies change.

Merely checking that the request is *still eligible* is insufficient. Suppose compiled addition code captured operands 3 and 5. Updating the second operand to 6 leaves word addition eligible, but executing the old code yields 8 instead of 9. Revalidating the actual dependencies selects new code. Changing an unrelated coordinate permits reuse even when the surrounding world changed.

The planner's reads are optimization evidence, not silently observable source effects. Definition revisions, owner identities, scope and active profiles must enter the fixed planner identity or its read keys whenever relevant. An unqualified hash or a global admission Boolean does not establish this law.

3 Interleaving is preserved, not removed

`InterleavedProtocolRegions` distinguishes execution pages from environment actions. A boundary supplies actual and reference state updates and proves their local decoding square. It preserves capability indices; capability-changing actions require a protocol that describes those changes. Environment updates retain the client continuation and any returned value. They do not rewind the invocation or restart a completed client.

The history theorem preserves

$$(\text{logical operations used},\ \text{decoded outcome with full residual state})$$

through every finite sequence of pages and boundaries. Adjacent pages compose by the existing exact resumption law. An environment action between pages is not erased by that law.

An additional controller may choose its next action from its control state and the current decoded reference receipt. Arbitrary bounded runs of this semantic-adaptive controller preserve the same receipt and controller state. The theorem does not justify an unqualified rewrite of a scheduler that observes private physical details. Logical charges count protocol operations; they do not assert equality of native work or optimization overhead.

`RegionDependencyStability` connects read certificates with the existing rely–guarantee claims. Its assertion is agreement with captured reads, its rely preserves those coordinates, and its guarantee permits writes only to a declared set. Cross read/write disjointness proves coexistence. Relevant changes are not globally forbidden: they instead invalidate reuse and demand validation or replanning. A concrete overlapping writer fails coexistence.

These are interleaving and dependency models. They do not prove that a C implementation provides the required atomic access or publication boundary. They also do not make a whole saturation run indivisible, or infer fairness from a scheduler that may repeatedly select zero work.

4 A nontrivial word implementation

`LiveWordRegions` instantiates the protocol with natural-valued registers and requests to read, load, add or subtract. The reference uses unbounded naturals. The planner produces code containing checked UInt64 operands. The executor uses actual word addition and, for truncated natural subtraction, word subtraction when ordered and zero otherwise. It does not invoke the reference operation to produce its fast result.

Addition is admitted exactly when the sum fits the word range. Subtraction in this two-word region is admitted exactly when both operands fit. These are complete characterizations of the declared implementation domains, not a globally optimal arithmetic representation theorem. Large values in unread registers do not prevent admission, and the full register state is retained. Overflow and unsupported operand ranges select exact fallback.

The executable control is a reply-dependent free protocol tree. Write $W = 2^{64}$. Starting from otherwise zero registers, it performs:

Request	Reply	Selected route
load $r_0 \leftarrow W + 7$	$W + 7$	reference
load $r_1 \leftarrow W + 3$	$W + 3$	reference
subtract $r_0 \leftarrow r_0 - r_1$	4	reference
load $r_2 \leftarrow$ preceding reply	4	word
add $r_0 \leftarrow r_0 + r_2$	8	word
load $r_1 \leftarrow W - 1$	$W - 1$	word
add $r_0 \leftarrow r_0 + r_1$	$W + 7$	reference
subtract $r_0 \leftarrow r_0 - r_1$	8	reference
add $r_0 \leftarrow r_0 + r_2$	12	word

The mode trace is an optimization receipt, not an added source answer. The generic theorem covers every allowance; the finite control additionally checks these actual replies and routes. A suspended prefix resumes into the word region without replay. An intervening relevant write changes the next admission; an unrelated write leaves the answers and route choices unchanged while its updated register remains retained.

5 Coordinate transport and controls

`LiveRegionRenaming` proves a concrete naturality result for bijective register renaming. Reference updates and native execution commute with the coordinate map. Code selection is identical; the ordered read certificate maps its keys. Actual partial attempts commute, including refusal. This is stronger than comparing two final numbers, but is not a blanket theorem for source alpha-equivalence across arbitrary variable frames. A noninjective name collapse changes readings and can change admission.

Further controls distinguish stale operands within an eligible region, overflow, a stale negative decision, repeated reads, unrelated writes, discarded request identity, and unchecked word wraparound. Arbitrarily many zero-allowance pages retain a paused computation rather than certify completion. These controls do not seal eventual completion of the separate O0 controller.

6 Qualification and remaining bridge

The six mathematical modules and joined audit are checked with the required Lean procedure. Owner-module manifests inventory generated helpers and primitive axiom dependencies, with compiler runtime counterparts separate. Only the standard Lean axioms are permitted; there are no admissions or added axioms in the qualified package.

The remaining production bridge is source-meaningful region extraction: which presented computation is a request, what its replies and observations are, and where interruption and interference are permitted. Real native state, ownership, atomicity, metagraph indexing and cost correspondence are separate obligations. The current work supplies reusable laws for that bridge without editing the runtime or replacing the general evaluator.